



हां, यही है Redux टूलकिट। ...
TXT 617.1KB

make notes for me out of this transcript explain everything properly

Here are comprehensive notes based on the provided transcript, explaining Redux Toolkit from the ground up and detailing the implementation of a Media Search Application.

Part 1: Core Concepts of Redux & Redux Toolkit

This section covers the "why" and "what" of Redux Toolkit, explaining the problems it solves and its fundamental architecture.

1. Why Redux? The Problem with React's State Management

- **React's Purpose:** React is a library for building UIs. Its key advantage is performance, thanks to the Virtual DOM, which allows for efficient UI updates.
- **The State Management Problem:** As applications grow, managing "state" (data that can change over time) becomes complex.
 - **State:** The current value of a variable or piece of data in your application (e.g., a username, a counter value, a theme setting).
 - **Prop Drilling:** In React, data is passed down from parent to child components via "props". To get data from a top-level component to a deeply nested one, you must pass it through every intermediate component. This is called "prop drilling."
 - **The Issue:** Prop drilling is cumbersome, makes code hard to read, and is difficult to manage in large applications. If a deeply nested component needs to update a value that a top-level component holds, it's very complicated.

2. The Redux Solution: Centralized State

- **The Core Idea:** Instead of passing data around, Redux creates a single, centralized "store" where all application state lives.
- **Analogy:** Think of a central board where you write down a shared password (like a Netflix password). Instead of passing it from person to person (prop drilling), everyone can just go to the board and access it.

- **Original Redux:** The first version of Redux solved the problem of centralized state management. However, it was known for being complex, having a lot of boilerplate code, and being difficult to set up.
- **Redux Toolkit (RTK): The Hero of the Story:** RTK is the modern, official, and recommended way to write Redux logic. It's like "Express for Node.js"—it simplifies the original, more complex tool. It drastically reduces boilerplate and makes Redux easier to use and understand.

3. Redux Toolkit Architecture Explained

The architecture is a unidirectional data flow, which can be broken down into these key parts:

1. **UI Layer:** The user interface (buttons, forms, etc.) that the user interacts with. For example, a "Change Theme" button.
2. **Event Handler:** When a user interacts with the UI (e.g., clicks the button), an event handler function is triggered. This function contains the logic for what to do next.
3. **Action:** The event handler **dispatches an action**. An action is a simple JavaScript object that describes *what happened* (e.g., `{ type: 'theme/changeTheme' }`). It's a "message" sent to the store.
4. **Redux Store:** This is the central warehouse that holds all the application's state. It's the single source of truth.
5. **Reducers:** The store contains one or more "reducer" functions. A reducer is a pure function that takes the current `state` and an `action`, and returns a *new state*.
 - The reducer for a specific feature (like `themeReducer`) listens for its specific actions (like `'theme/changeTheme'`).
 - When it receives the action, it updates the state in the store (e.g., changes the theme value from 'light' to 'dark').
6. **UI Update:** Because the state in the store has changed, the UI components that are connected to that state automatically re-render with the new value.

Key Terms:

- **Store:** A centralized container holding the complete state of the app.
- **Action:** A plain object describing an event that occurred.
- **Reducer:** A function that determines how the state changes in response to an action.
- **Dispatch:** The method used to send an action to the store.

- **Selector:** A function used to read a specific piece of data from the store's state.

Part 2: A Simple Counter Example (Learn by Doing)

This part walks through a basic counter app to demonstrate how to implement RTK concepts.

Step 1: Project Setup & Installation

1. Create a React project:

```
bash
```

```
npm create vite@latest ./ -- --template react
```

(Use `@latest` for the newest version, but a stable one is also fine).

2. Install Redux Toolkit and React-Redux:

```
bash
```

```
npm install @reduxjs/toolkit react-redux
```

3. Clean up boilerplate: Remove unnecessary boilerplate from `App.jsx` and `index.css`.

Step 2: Create the Redux Store

The store is the central place where all state lives.

1. Create a folder named `redux` inside `src`.
2. Inside `redux`, create a file `store.js`.
3. Use `configureStore` from Redux Toolkit to create the store. Initially, it will have an empty reducer object.

```
javascript
```

```
// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';

export const store = configureStore({
  reducer: {}, // Reducers will be added here later
});
```

Step 3: Create a "Slice"

A "slice" is a collection of Redux logic for a single feature of your application. The counter feature will have its own slice.

1. Create a folder `features` inside `src`.
2. Inside `features`, create a `counter` folder.
3. Inside `counter`, create a `counterSlice.js` file.
4. Use `createSlice` to define the slice. It requires:
 - `name`: A unique name for the slice.
 - `initialState`: The starting state for that feature.
 - `reducers`: An object containing functions that define how the state can be updated. These functions are the "actions".

javascript

```
// src/features/counter/counterSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  value: 0,
};

export const counterSlice = createSlice({
  name: 'counter',
  initialState,
  reducers: {
    increment: (state) => {
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    },
  },
});

// Export the actions
export const { increment, decrement, incrementByAmount } = counterSlice.actions;
```

```
// Export the reducer
export default counterSlice.reducer;
```

Note: RTK uses Immer.js internally, which allows you to write "mutating" logic like `state.value += 1`. It translates this into an immutable update behind the scenes.

Step 4: Add the Slice Reducer to the Store

Now, connect the slice to the main store.

```
javascript

// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from '../features/counter/counterSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer, // Add the counter reducer here
  },
});
```

Step 5: Provide the Store to the React App

Wrap your entire application with the `<Provider>` component from `react-redux` to make the store accessible to all components.

```
jsx

// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';
import { store } from '../redux/store';
import { Provider } from 'react-redux';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

Step 6: Use State and Actions in a Component

In your `App.jsx` component, use the `useSelector` hook to read data and `useDispatch` to send actions.

```
jsx

// src/App.jsx
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, incrementByAmount } from '../features/counter/counterSlice';
import { useState } from 'react';

function App() {
  // Read the current counter value from the store
  const count = useSelector((state) => state.counter.value);
  // Get the dispatch function
  const dispatch = useDispatch();

  const [amount, setAmount] = useState(5);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}>Increment</button>
      <button onClick={() => dispatch(decrement())}>Decrement</button>
      <input
        type="number"
        value={amount}
        onChange={(e) => setAmount(Number(e.target.value))}
      />
      <button onClick={() => dispatch(incrementByAmount(amount))}>
        Increment by {amount}
      </button>
    </div>
  );
}

export default App;
```

Part 3: Media Search Application Project

This project applies the concepts of Redux Toolkit to build a real-world application that searches for photos, videos, and GIFs.

1. Project Setup & Folder Structure

- **Create Project:**

```
bash
```

```
npm create vite@latest ./ -- --template react
```

- **Install Dependencies:**

```
bash
```

```
npm install @reduxjs/toolkit react-redux axios react-router-dom react-toastify
```

- `axios` : For making API requests.
- `react-router-dom` : For navigation between pages.
- `react-toastify` : For showing notifications.
- **Setup Tailwind CSS (Optional but recommended):** Follow the official Tailwind CSS guide for Vite.
- **API Keys:** Obtain API keys from Unsplash, Pexels, and Tenor. Store them in a `.env` file in the root of your project (NOT inside `src`).

```
text
```

```
VITE_UNSPASH_KEY=your_unsplash_key  
VITE_PEXELS_KEY=your_pexels_key  
VITE_TENOR_KEY=your_tenor_key
```

- **Folder Structure (`src`):**

```
text
```

```
src/  
├─ api/  
│   └─ mediaApi.js  
├─ components/  
│   ├── Navbar.jsx  
│   ├── ResultCard.jsx  
│   ├── ResultGrid.jsx  
│   └─ SearchBar.jsx
```

```

|   └─ Tabs.jsx
└─ pages/
|   └─ CollectionPage.jsx
|   └─ HomePage.jsx
└─ redux/
|   └─ features/
|       └─ searchSlice.js
|       └─ collectionSlice.js
└─ store.js
└─ App.jsx
└─ main.jsx
└─ index.css

```

2. Create the API Fetching Logic (mediaApi.js)

This file handles all API calls and normalizes the data from different APIs into a consistent format.

javascript

```

// src/api/mediaApi.js
import axios from 'axios';

const UNSPLASH_KEY = import.meta.env.VITE_UNSPASH_KEY;
const PEXELS_KEY = import.meta.env.VITE_PEXELS_KEY;
const TENOR_KEY = import.meta.env.VITE_TENOR_KEY;

// Fetch Photos from Unsplash
export const fetchPhotos = async (query, page = 1, perPage = 20) => {
  const response = await axios.get('https://api.unsplash.com/search/photos', {
    params: { query, page, per_page: perPage },
    headers: { Authorization: `Client-ID ${UNSPLASH_KEY}` },
  });
  // Normalization: Map the response to a consistent structure
  return response.data.results.map((item) => ({
    id: item.id,
    type: 'photo',
    title: item.alt_description,
    thumbnail: item.urls.small,
    src: item.urls.full,
    url: item.links.html, // Link to the original page
  }));
};

// Fetch Videos from Pexels
export const fetchVideos = async (query, perPage = 15) => {

```

```

const response = await axios.get('https://api.pexels.com/videos/search', {
  params: { query, per_page: perPage },
  headers: { Authorization: PEXELS_KEY },
});
// Normalization
return response.data.videos.map((item) => ({
  id: item.id,
  type: 'video',
  title: item.user.name, // Or another suitable title
  thumbnail: item.image,
  src: item.video_files[0].link, // Link to the video file
  url: item.url,
})));
};

// Fetch GIFs from Tenor
export const fetchGifs = async (query, limit = 20) => {
  const response = await axios.get('https://tenor.googleapis.com/v2/search', {
    params: {
      q: query,
      key: TENOR_KEY,
      limit: limit,
    },
  });
  // Normalization
  return response.data.results.map(item => ({
    id: item.id,
    type: 'gif',
    title: item.title || 'GIF',
    thumbnail: item.media_formats.tinygif.url,
    src: item.media_formats.gif.url,
    url: item.itemurl,
  })));
};

```

3. Create Redux Slices

searchSlice.js : Manages the state for the search feature.

javascript

```

// src/redux/features/searchSlice.js
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  query: '',
  activeTab: 'photos', // 'photos', 'videos', 'gifs'

```

```

    results: [],
    loading: false,
    error: null,
  };

const searchSlice = createSlice({
  name: 'search',
  initialState,
  reducers: {
    setQuery: (state, action) => { state.query = action.payload; },
    setActiveTab: (state, action) => { state.activeTab = action.payload; },
    setLoading: (state, action) => { state.loading = action.payload; state.error = null; },
    setResults: (state, action) => { state.results = action.payload; state.loading = false; },
    setError: (state, action) => { state.error = action.payload; state.loading = false; },
    clearResults: (state) => { state.results = []; },
  },
});

export const { setQuery, setActiveTab, setLoading, setResults, setError, clearResults } =
searchSlice.actions;
export default searchSlice.reducer;

```

collectionSlice.js : Manages the user's saved collection.

javascript

```

// src/redux/features/collectionSlice.js
import { createSlice } from '@reduxjs/toolkit';
import { toast } from 'react-toastify';

// Initialize state from localStorage
const initialState = {
  items: JSON.parse(localStorage.getItem('collection')) || [],
};

const collectionSlice = createSlice({
  name: 'collection',
  initialState,
  reducers: {
    addCollection: (state, action) => {
      const alreadyExists = state.items.find(item => item.id === action.payload.id);
      if (!alreadyExists) {
        state.items.push(action.payload);
        localStorage.setItem('collection', JSON.stringify(state.items));
      }
    },
  },
});

```

```

        toast.success('Added to Collection!');
    }
},
removeCollection: (state, action) => {
    state.items = state.items.filter(item => item.id !== action.payload);
    localStorage.setItem('collection', JSON.stringify(state.items));
    toast.error('Removed from Collection!');
},
clearCollection: (state) => {
    state.items = [];
    localStorage.removeItem('collection');
    toast.info('Collection Cleared!');
}
},
});

export const { addCollection, removeCollection, clearCollection } = collectionSlice.actions;
export default collectionSlice.reducer;

```

4. Configure the Store

Combine the reducers in the store.

javascript

```

// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import searchReducer from '../features/searchSlice';
import collectionReducer from '../features/collectionSlice';

export const store = configureStore({
    reducer: {
        search: searchReducer,
        collection: collectionReducer,
    },
});

```

5. Build Components

- `SearchBar.jsx` : An input form that dispatches `setQuery` on submit.
- `Tabs.jsx` : Buttons for 'Photos', 'Videos', 'GIFs'. It uses `useSelector` to get the `activeTab` and `useDispatch` to call `setActiveTab` on click.
- `ResultCard.jsx` : Displays a single media item. It has a "Save" button that dispatches the `addCollection` action with the item's data.

- `ResultGrid.jsx` : Uses `useSelector` to get `results` , `loading` , and `error` from the `search` state. It fetches data in a `useEffect` hook whenever the `query` or `activeTab` changes. It then maps over the `results` array to render a `ResultCard` for each item.
- `Navbar.jsx` : Contains links to the Home page (`/`) and the Collection page (`/collection`).

6. Set up Routing and Pages

- `App.jsx` : Sets up the routes using `react-router-dom` .

`jsx`

```
// src/App.jsx
import { Routes, Route } from 'react-router-dom';
import HomePage from './pages/HomePage';
import CollectionPage from './pages/CollectionPage';
import Navbar from './components/Navbar';

function App() {
  return (
    <div className="bg-gray-950 min-h-screen text-white">
      <Navbar />
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/collection" element={<CollectionPage />} />
      </Routes>
    </div>
  );
}
export default App;
```

- `HomePage.jsx` : Renders `SearchBar` , `Tabs` , and `ResultGrid` .
- `CollectionPage.jsx` : Uses `useSelector` to get the `items` from the `collection` state. It maps over them to display the saved items, each with a "Remove" button that dispatches `removeCollection` . It also has a "Clear All" button that dispatches `clearCollection` .

Key Takeaways

- **Redux Toolkit Simplifies Redux:** It reduces boilerplate, making state management easier and more intuitive.

- **Centralized State is Key:** The store acts as a single source of truth, solving the prop-drilling problem.
- **Unidirectional Data Flow:** The cycle of UI → dispatch(Action) → Reducer → Store → UI is the core pattern.
- **Slices for Features:** `createSlice` is a powerful tool for organizing Redux logic by feature.
- **Hooks for React Integration:** `useSelector` (for reading state) and `useDispatch` (for sending actions) are the primary ways to connect React components to the Redux store.
- **Normalization is Crucial:** When working with multiple APIs, normalize the data into a consistent shape to simplify rendering and state management.
- **Persistence with `localStorage`:** The `collectionSlice` demonstrates how to persist state across browser sessions using `localStorage`.

give me interview questions with answer

Redux Toolkit Interview Questions & Answers

Based on the transcript, here are comprehensive interview questions with detailed answers covering Redux Toolkit concepts and the Media Search Application project.

Section 1: Redux Fundamentals

Q1: What is Redux, and why do we need it in React applications?

Answer:

Redux is a standalone state management library that is **not** part of React. It solves the problem of managing state in large applications.

The Problem it Solves:

- **Prop Drilling:** In React, data flows from parent to child via props. To pass data to a deeply nested component, you must pass it through every intermediate component. This is cumbersome and error-prone.
- **State Synchronization:** When multiple components need to share and update the same state, keeping them in sync becomes difficult.

The Redux Solution:

Redux creates a **centralized store** where all application state lives. Any component can access the state directly without prop drilling.

Analogy: Think of a central board where a shared password is written. Instead of passing it person-to-person (prop drilling), everyone goes to the board to access it.

Q2: What is Redux Toolkit, and how is it different from plain Redux?

Answer:

Redux Toolkit (RTK) is the **official, modern, and recommended** way to write Redux logic. It is essentially "Redux 2.0."

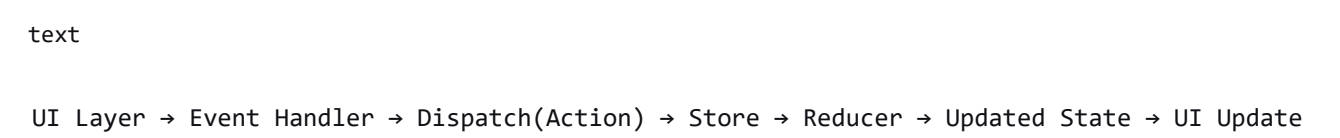
Plain Redux	Redux Toolkit
Complex, verbose syntax	Simple, concise syntax
Long boilerplate code	Minimal boilerplate
Difficult setup	Easy setup with <code>configureStore</code>
Manual immutable updates	Uses Immer.js for "mutating" syntax that becomes immutable
Separate action creators, action types, reducers	All combined in <code>createSlice</code>

Key Improvement: RTK reduces boilerplate by combining action types, action creators, and reducers into a single `createSlice` function.

Q3: Explain the Redux architecture and data flow.

Answer:

Redux follows a **unidirectional data flow**:



Step-by-Step:

1. **UI Layer:** User interacts with the UI (e.g., clicks a "Change Theme" button).
2. **Event Handler:** The click triggers an event handler function.
3. **Action Dispatch:** The event handler dispatches an action—a plain object describing *what happened* (e.g., `{ type: 'theme/changeTheme' }`).
4. **Store:** The action reaches the Redux store (central state container).
5. **Reducer:** The store calls the appropriate reducer function, which takes the current state and action, and returns a **new state**.
6. **State Update:** The store updates its state.
7. **UI Re-render:** Components subscribed to that state automatically re-render with the new value.

Key Terms:

- **Store:** Central container holding the complete app state.
- **Action:** Plain object describing an event.
- **Reducer:** Pure function that determines state changes.
- **Dispatch:** Method to send an action to the store.
- **Selector:** Function to read specific data from the store.

Q4: What is a "slice" in Redux Toolkit?

Answer:

A **slice** is a collection of Redux logic for a **single feature** of your application. It's created using `createSlice` and contains:

1. `name` : A unique identifier for the slice (e.g., 'counter', 'search', 'collection').
2. `initialState` : The starting state for that feature.
3. `reducers` : An object containing functions that define how state can be updated. These become **actions**.

Example:

```
javascript
```

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action) => { state.value += action.payload; },
  },
});

export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

Why "Slice"? It represents a "slice" of the overall application state. For example, a shopping app might have `cartSlice`, `userSlice`, and `productSlice`.

Q5: Why can we write "mutating" code like `state.value += 1` in Redux Toolkit?

Answer:

Redux Toolkit uses **Immer.js** internally. Immer allows you to write code that *looks* like it's mutating state directly, but behind the scenes, it creates a **new immutable state** based on your changes.

How it works:

- Immer creates a "draft" copy of the state.
- You make changes to the draft (e.g., `state.value += 1`).
- Immer compares the draft to the original and produces a new, immutable state.

Without Immer (plain Redux):

javascript

```
return { ...state, value: state.value + 1 }; // Manual spread
```

With Immer (RTK):

javascript

```
state.value += 1; // Looks mutating, but is immutable
```

Section 2: Redux Toolkit Implementation

Q6: How do you set up a Redux store with Redux Toolkit?

Answer:

Step 1: Install dependencies

bash

```
npm install @reduxjs/toolkit react-redux
```

Step 2: Create the store

javascript

```
// src/redux/store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from '../features/counter/counterSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});
```

Step 3: Provide the store to React

jsx

```
// src/main.jsx
import { Provider } from 'react-redux';
import { store } from '../redux/store';

ReactDOM.createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Key Point: `configureStore` automatically sets up the Redux DevTools Extension and includes `redux-thunk` for async logic.

Q7: What is the difference between `useSelector` and `useDispatch` ?

Answer:

<code>useSelector</code>	<code>useDispatch</code>
Reads data from the store	Sends actions to the store
Returns a piece of state	Returns the dispatch function
Subscribes component to state changes	Triggers state updates via reducers

Example:

```
jsx

import { useSelector, useDispatch } from 'react-redux';
import { increment } from './counterSlice';

function Counter() {
  // Read state
  const count = useSelector((state) => state.counter.value);

  // Get dispatch function
  const dispatch = useDispatch();

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => dispatch(increment())}>Increment</button>
    </div>
  );
}
```

`useSelector` re-renders the component whenever the selected state changes.

`useDispatch` is used to trigger actions that update the state.

Q8: How do you handle asynchronous operations (API calls) in Redux Toolkit?

Answer:

There are two main approaches:

Approach 1: Direct API calls in components (used in the project)

jsx

```
useEffect(() => {
  const getData = async () => {
    dispatch(setLoading(true));
    try {
      let data;
      if (activeTab === 'photos') data = await fetchPhotos(query);
      else if (activeTab === 'videos') data = await fetchVideos(query);
      else data = await fetchGifs(query);
      dispatch(setResults(data));
    } catch (error) {
      dispatch(setError(error.message));
    }
  };
  getData();
}, [query, activeTab]);
```

Approach 2: Using createAsyncThunk (advanced)

javascript

```
export const fetchPhotosAsync = createAsyncThunk(
  'search/fetchPhotos',
  async (query) => {
    const response = await fetchPhotos(query);
    return response;
  }
);

// In the slice:
extraReducers: (builder) => {
  builder
    .addCase(fetchPhotosAsync.pending, (state) => { state.loading = true; })
    .addCase(fetchPhotosAsync.fulfilled, (state, action) => {
      state.loading = false;
      state.results = action.payload;
    })
}
```

```

    .addCase(fetchPhotosAsync.rejected, (state, action) => {
      state.loading = false;
      state.error = action.error.message;
    });
  }
}

```

Note: The transcript mentions that `createAsyncThunk` was **not** covered in detail and is planned for future videos.

Q9: How do you persist Redux state to localStorage?

Answer:

You can persist state by reading from and writing to `localStorage` inside your slice's reducers.

Initial State from localStorage:

javascript

```

const initialState = {
  items: JSON.parse(localStorage.getItem('collection')) || [],
};

```

Writing to localStorage in reducers:

javascript

```

addCollection: (state, action) => {
  const alreadyExists = state.items.find(item => item.id === action.payload.id);
  if (!alreadyExists) {
    state.items.push(action.payload);
    localStorage.setItem('collection', JSON.stringify(state.items));
  }
},

removeCollection: (state, action) => {
  state.items = state.items.filter(item => item.id !== action.payload);
  localStorage.setItem('collection', JSON.stringify(state.items));
},

clearCollection: (state) => {
  state.items = [];
}

```

```
localStorage.removeItem('collection');  
}
```

Why this works: When the app loads, the initial state is populated from localStorage. Every subsequent change updates both Redux state and localStorage.

Section 3: Project-Specific Questions

Q10: What is data normalization in the context of API responses, and why is it important?

Answer:

Normalization is the process of transforming data from different APIs into a **consistent, unified structure**.

The Problem: When fetching from multiple APIs (Unsplash, Pexels, Tenor), each returns data in a different shape:

- **Unsplash:** { id, alt_description, urls: { small, full }, links: { html } }
- **Pexels:** { id, user: { name }, image, video_files: [{ link }], url }
- **Tenor:** { id, title, media_formats: { tinygif: { url }, gif: { url } }, itemurl }

The Solution: Normalize all responses to a common shape:

javascript

```
{  
  id: 'unique-id',  
  type: 'photo' | 'video' | 'gif',  
  title: 'description',  
  thumbnail: 'small-image-url',  
  src: 'full-resolution-url',  
  url: 'link-to-original-page'  
}
```

Why it's important:

- **Simplifies rendering:** One `ResultCard` component can handle all media types.
- **Reduces bugs:** No need to check which API the data came from.

- **Easier state management:** The `results` array in Redux has a uniform structure.

Q11: How did you structure the Media Search Application's Redux store?

Answer:

The store has **two slices**, each managing a distinct feature:

javascript

```
export const store = configureStore({
  reducer: {
    search: searchReducer,      // Manages search functionality
    collection: collectionReducer, // Manages saved items
  },
});
```

1. searchSlice :

javascript

```
{
  query: '',          // User's search term
  activeTab: 'photos', // 'photos' | 'videos' | 'gifs'
  results: [],        // Array of normalized media items
  loading: false,     // Loading state
  error: null,        // Error message
}
```

2. collectionSlice :

javascript

```
{
  items: [], // Array of saved media items (from localStorage)
}
```

Why this structure?

- **Separation of concerns:** Search logic is separate from collection logic.
- **Scalability:** New features can be added as new slices.

- **Maintainability:** Each slice is self-contained with its own actions and reducers.

Q12: How does the tab switching mechanism work in the application?

Answer:

The tab switching uses Redux state and the `useSelector` / `useDispatch` hooks.

1. State: `activeTab` is stored in `searchSlice` :

javascript

```
initialState = {  
  activeTab: 'photos',  
  // ...  
}
```

2. Action: `setActiveTab` updates the state:

javascript

```
setActiveTab: (state, action) => {  
  state.activeTab = action.payload;  
}
```

3. Component (`Tabs.jsx`):

jsx

```
const Tabs = () => {  
  const dispatch = useDispatch();  
  const activeTab = useSelector((state) => state.search.activeTab);  
  const tabs = ['photos', 'videos', 'gifs'];  
  
  return (  
    <div className="flex gap-5">  
      {tabs.map((tab, idx) => (  
        <button  
          key={idx}  
          onClick={() => dispatch(setActiveTab(tab))}  
          className={`px-4 py-2 rounded ${  
            activeTab === tab ? 'bg-blue-700' : 'bg-gray-600'  
          }}`  
        >>  
      )}  
    >>  
  )  
}
```

```

    >
    {tab.toUpperCase()}
  </button>
  )}}
</div>
);
};

```

4. Data Fetching (ResultGrid.jsx):

jsx

```

useEffect(() => {
  const getData = async () => {
    if (activeTab === 'photos') data = await fetchPhotos(query);
    else if (activeTab === 'videos') data = await fetchVideos(query);
    else data = await fetchGifs(query);
    dispatch(setResults(data));
  };
  if (query) getData();
}, [query, activeTab]); // Re-fetches when tab changes

```

Flow: Click tab → dispatch setActiveTab → state updates → useEffect detects change → fetches new data → dispatches setResults → UI updates.

Q13: How do you prevent duplicate items from being added to the collection?

Answer:

The addCollection reducer checks if an item with the same id already exists before adding it.

javascript

```

addCollection: (state, action) => {
  const alreadyExists = state.items.find(
    item => item.id === action.payload.id
  );

  if (!alreadyExists) {
    state.items.push(action.payload);
    localStorage.setItem('collection', JSON.stringify(state.items));
  }
}

```

```

    toast.success('Added to Collection!');
  }
}

```

Explanation:

- `state.items.find()` returns the first item where `item.id === action.payload.id`.
- If `alreadyExists` is truthy, the item is already in the collection, so we skip adding it.
- If `alreadyExists` is undefined, the item is new, so we push it and save to `localStorage`.

Alternative: You could use `some()` for a boolean check:

javascript

```
const alreadyExists = state.items.some(item => item.id === action.payload.id);
```

Q14: How is routing implemented in the application?

Answer:

Routing uses `react-router-dom` with two main routes.

1. Setup in `main.jsx`:

jsx

```

import { BrowserRouter } from 'react-router-dom';

<BrowserRouter>
  <Provider store={store}>
    <App />
  </Provider>
</BrowserRouter>

```

2. Routes in `App.jsx`:

jsx

```

import { Routes, Route } from 'react-router-dom';
import HomePage from './pages/HomePage';
import CollectionPage from './pages/CollectionPage';

function App() {

```

```

return (
  <Routes>
    <Route path="/" element={<HomePage />} />
    <Route path="/collection" element={<CollectionPage />} />
  </Routes>
);
}

```

3. Navigation in Navbar.jsx :

jsx

```

import { Link } from 'react-router-dom';

<nav className="flex justify-between items-center p-5">
  <h2 className="text-2xl font-semibold">Media Search</h2>
  <div className="flex gap-5">
    <Link to="/">Search</Link>
    <Link to="/collection">Collection</Link>
  </div>
</nav>

```

Key Points:

- `BrowserRouter` wraps the entire app to enable routing.
- `Routes` contains all `Route` definitions.
- `Link` is used instead of `<a>` tags for client-side navigation (no page reload).

Q15: How do you handle loading and error states in the application?

Answer:

Loading and error states are managed in `searchSlice` and displayed conditionally in components.

State:

javascript

```

initialState = {
  loading: false,
  error: null,

```

```
// ...
}
```

Actions:

javascript

```
setLoading: (state, action) => {
  state.loading = action.payload;
  state.error = null; // Clear error when loading starts
},
setResults: (state, action) => {
  state.results = action.payload;
  state.loading = false; // Stop loading when results arrive
},
setError: (state, action) => {
  state.error = action.payload;
  state.loading = false; // Stop loading on error
},
```

Usage in ResultGrid.jsx :

jsx

```
const { results, loading, error } = useSelector((state) => state.search);

if (error) return <h1>Error: {error}</h1>;
if (loading) return <h1>Loading...</h1>;

return (
  <div>
    {results.map((item) => (
      <ResultCard key={item.id} item={item} />
    ))}
  </div>
);
```

Flow:

1. User searches → setLoading(true) → UI shows "Loading..."
2. API succeeds → setResults(data) → loading = false → UI shows results
3. API fails → setError(message) → loading = false → UI shows error

Section 4: Advanced & Conceptual Questions

Q16: What is prop drilling, and how does Redux solve it?

Answer:

Prop Drilling: Passing data from a parent component to a deeply nested child through multiple intermediate components that don't need the data themselves.

Example (without Redux):

```
jsx

// App has data
function App() {
  const user = { name: 'Sarthak' };
  return <Header user={user} />;
}

// Header doesn't need user, but must pass it down
function Header({ user }) {
  return <Navbar user={user} />;
}

// Navbar doesn't need user either
function Navbar({ user }) {
  return <Profile user={user} />;
}

// Only Profile actually needs the data
function Profile({ user }) {
  return <h1>{user.name}</h1>;
}
```

With Redux:

```
jsx

// Any component can access state directly
function Profile() {
  const user = useSelector((state) => state.user);
  return <h1>{user.name}</h1>;
}
```

How Redux Solves It:

- State is stored centrally in the Redux store.
- Any component can access any piece of state via `useSelector`.
- No need to pass props through intermediate components.

Q17: What are the advantages of using Redux Toolkit over Context API?

Answer:

Feature	Context API	Redux Toolkit
Purpose	Prop drilling solution	Full state management
Performance	Re-renders all consumers on any change	Optimized with selectors
DevTools	No built-in DevTools	Excellent Redux DevTools
Middleware	Manual setup	Built-in (thunk, etc.)
Async Logic	Manual	<code>createAsyncThunk</code> support
Boilerplate	Low	Low (with RTK)
Scalability	Good for small apps	Excellent for large apps
Debugging	Difficult	Time-travel debugging

When to use Context API:

- Small applications
- Simple state (e.g., theme, language)
- Infrequent updates

When to use Redux Toolkit:

- Large applications
- Complex state logic
- Frequent updates

- Need for DevTools and middleware

Q18: Explain the concept of "immutability" in Redux.

Answer:

Immutability means state is never modified directly. Instead, a **new state object** is created with the desired changes.

Why Immutability?

- **Predictability:** State changes are explicit and traceable.
- **Performance:** Redux can compare old and new state by reference to determine if re-rendering is needed.
- **Debugging:** Time-travel debugging requires a history of states.

Without Immer (plain Redux):

```
javascript

// WRONG - mutating state directly
state.value = 10;

// CORRECT - creating a new state
return { ...state, value: 10 };
```

With Redux Toolkit (Immer):

```
javascript

// Looks mutating, but Immer handles immutability
state.value = 10;
```

How Immer Works:

1. Creates a "draft" copy of the state.
2. You modify the draft.
3. Immer compares the draft to the original and produces a new immutable state.

Q19: What is the role of `configureStore` in Redux Toolkit?

Answer:

`configureStore` is a convenience function that simplifies store setup.

What it does:

1. **Combines reducers:** Automatically calls `combineReducers` if you pass an object of reducers.
2. **Adds middleware:** Includes `redux-thunk` by default for async logic.
3. **Sets up DevTools:** Enables Redux DevTools Extension automatically.
4. **Provides defaults:** Sensible defaults for development and production.

Example:

javascript

```
import { configureStore } from '@reduxjs/toolkit';
import searchReducer from '../features/searchSlice';
import collectionReducer from '../features/collectionSlice';

export const store = configureStore({
  reducer: {
    search: searchReducer,
    collection: collectionReducer,
  },
});
```

Equivalent in plain Redux (more verbose):

javascript

```
import { createStore, combineReducers, applyMiddleware } from 'redux';
import thunk from 'redux-thunk';
import { composeWithDevTools } from 'redux-devtools-extension';

const rootReducer = combineReducers({
  search: searchReducer,
  collection: collectionReducer,
});

const store = createStore(
  rootReducer,
```

```
    composeWithDevTools(applyMiddleware(thunk))
  );
```

Key Point: `configureStore` reduces boilerplate and provides best-practice defaults.

Q20: How would you debug a Redux application?

Answer:

1. Redux DevTools Extension:

- Install the browser extension (Chrome/Firefox).
- View every dispatched action and the resulting state.
- **Time-travel debugging:** Jump to any previous state.
- Inspect the difference between states.

2. Console Logging:

javascript

```
dispatch(setResults(data));
console.log('Results:', data);
```

3. Middleware for Logging:

javascript

```
const logger = (store) => (next) => (action) => {
  console.log('Dispatching:', action);
  const result = next(action);
  console.log('New State:', store.getState());
  return result;
};

configureStore({
  reducer: { ... },
  middleware: (getDefaultMiddleware) => getDefaultMiddleware().concat(logger),
});
```

4. Common Issues & Solutions:

Issue	Cause	Solution
State not updating	Reducer not returning new state	Check reducer logic
Component not re-rendering	Selector returning same reference	Use <code>useSelector</code> correctly
Action not dispatched	Missing <code>useDispatch</code>	Import and use <code>useDispatch</code>
Provider not wrapping app	Store not accessible	Wrap app with <code><Provider store={store}></code>

Section 5: Scenario-Based Questions

Q21: You need to add a "Remove from Collection" feature. How would you implement it?

Answer:

Step 1: Add action to `collectionSlice` :

javascript

```
removeCollection: (state, action) => {
  state.items = state.items.filter(item => item.id !== action.payload);
  localStorage.setItem('collection', JSON.stringify(state.items));
  toast.error('Removed from Collection!');
}
```

Step 2: Export the action:

javascript

```
export const { addCollection, removeCollection, clearCollection } = collectionSlice.actions;
```

Step 3: Use in component (`CollectionCard.jsx`):

jsx

```
import { useDispatch } from 'react-redux';
import { removeCollection } from '../redux/features/collectionSlice';

function CollectionCard({ item }) {
  const dispatch = useDispatch();

  return (
    <div>
      <img src={item.thumbnail} alt={item.title} />
      <button onClick={() => dispatch(removeCollection(item.id))}>
        Remove
      </button>
    </div>
  );
}
```

Explanation:

- The reducer filters out the item with the matching `id`.
- `localStorage` is updated to reflect the change.
- A toast notification confirms the removal.

Q22: How would you implement a "Clear All" feature for the collection?

Answer:

Step 1: Add action to `collectionSlice` :

javascript

```
clearCollection: (state) => {
  state.items = [];
  localStorage.removeItem('collection');
  toast.info('Collection Cleared!');
}
```

Step 2: Export the action:

javascript

```
export const { addCollection, removeCollection, clearCollection } = collectionSlice.actions;
```

Step 3: Use in CollectionPage.jsx :

jsx

```
import { useDispatch, useSelector } from 'react-redux';
import { clearCollection } from '../redux/features/collectionSlice';

function CollectionPage() {
  const dispatch = useDispatch();
  const collection = useSelector((state) => state.collection.items);

  return (
    <div>
      <div className="flex justify-between">
        <h2>Your Collection</h2>
        <button
          onClick={() => dispatch(clearCollection())}
          className="bg-red-600 px-5 py-2 rounded"
        >
          Clear Collection
        </button>
      </div>
      {collection.map((item) => (
        <CollectionCard key={item.id} item={item} />
      ))}
    </div>
  );
}
```

Explanation:

- The reducer sets `items` to an empty array.
- `localStorage.removeItem('collection')` deletes the persisted data.
- The UI re-renders with an empty collection.

Q23: What happens if you forget to add a reducer to the store?

Answer:

If you forget to add a reducer to the store, you'll get an error like:

```
text
```

```
Error: Cannot read properties of undefined (reading 'value')
```

Why?

- The component tries to access `state.search.results`, but `state.search` is undefined because the reducer wasn't registered.
- The store only contains reducers that were explicitly added.

Example:

```
javascript
```

```
// store.js - MISSING searchReducer
export const store = configureStore({
  reducer: {
    collection: collectionReducer, // search is missing!
  },
});

// Component
const results = useSelector((state) => state.search.results); // ERROR!
```

Solution:

Always register all reducers in the store:

```
javascript
```

```
export const store = configureStore({
  reducer: {
    search: searchReducer,
    collection: collectionReducer,
  },
});
```

Debugging Tip: Use Redux DevTools to inspect the store's state shape and verify all reducers are registered.

Q24: How would you handle API errors gracefully in the application?

Answer:

1. In the API function:

javascript

```
export const fetchPhotos = async (query) => {
  try {
    const response = await axios.get('...', { ... });
    return response.data.results.map(...);
  } catch (error) {
    throw new Error(error.response?.data?.message || 'Failed to fetch photos');
  }
};
```

2. In the component:

jsx

```
useEffect(() => {
  const getData = async () => {
    dispatch(setLoading(true));
    try {
      const data = await fetchPhotos(query);
      dispatch(setResults(data));
    } catch (error) {
      dispatch(setError(error.message));
    }
  };
  if (query) getData();
}, [query, activeTab]);
```

3. In the slice:

javascript

```
setError: (state, action) => {
  state.error = action.payload;
  state.loading = false;
}
```

4. In the UI:

jsx

```
if (error) return (  
  <div className="text-red-500 text-center">  
    <h1>Error: {error}</h1>  
    <button onClick={() => window.location.reload()}>Retry</button>  
  </div>  
);
```

5. Toast Notification (optional):

javascript

```
catch (error) {  
  dispatch(setError(error.message));  
  toast.error(error.message);  
}
```

Best Practices:

- Always use try/catch for async operations.
- Provide meaningful error messages.
- Offer a retry mechanism.
- Log errors for debugging.

Q25: How would you optimize performance in a Redux application?

Answer:

1. Use Reselect for Memoized Selectors:

javascript

```
import { createSelector } from '@reduxjs/toolkit';  
  
const selectResults = (state) => state.search.results;  
  
export const selectPhotoResults = createSelector(  
  [selectResults],  
  (results) => results.filter(item => item.type === 'photo')  
);
```

2. Avoid Unnecessary Re-renders:

- Use `useSelector` with specific selectors (not the entire state).
- Split components so only relevant parts re-render.

3. Normalize State:

- Store items in a flat structure ({ `byId`: {}, `allIds`: [] }) for efficient lookups.

4. Use `React.memo` :

```
jsx

const ResultCard = React.memo(({ item }) => {
  // Component only re-renders if `item` changes
});
```

5. Debounce Search Input:

```
jsx

const debouncedSearch = useMemo(
  () => debounce((value) => dispatch(setQuery(value)), 500),
  []
);
```

6. Lazy Loading:

- Load images/videos only when they enter the viewport.

7. Pagination:

- Fetch data in pages instead of all at once.

Quick Reference: Key Redux Toolkit APIs

API	Purpose
<code>configureStore</code>	Creates the Redux store with sensible defaults
<code>createSlice</code>	Creates a slice with reducers and actions

API	Purpose
<code>createAsyncThunk</code>	Handles async operations (not covered in depth)
<code>useSelector</code>	Reads state from the store
<code>useDispatch</code>	Dispatches actions to the store
<code>Provider</code>	Makes the store available to React components
<code>createSelector</code>	Creates memoized selectors (Reselect)

Summary: Key Concepts to Remember

1. **Redux solves prop drilling** by centralizing state in a store.
2. **Redux Toolkit is Redux 2.0**—simpler, less boilerplate, best practices built-in.
3. **Slices organize logic by feature** (e.g., `searchSlice`, `collectionSlice`).
4. **Immer allows "mutating" syntax** that produces immutable state.
5. `useSelector` **reads state**; `useDispatch` **sends actions**.
6. **Normalization** unifies data from different APIs.
7. **localStorage** persists state across sessions.
8. **Loading/Error states** improve UX during async operations.
9. **Routing** with `react-router-dom` enables multi-page navigation.
10. **Debug with Redux DevTools** for time-travel debugging.